

How informative is the Order Book Beyond the Best Levels? Machine Learning Perspective

Dat Thanh Tran^{1*}, Juho Kanniainen¹, Alexandros Iosifidis²

¹*Department of Computing Sciences, Tampere University, Finland*

²*Department of Electrical and Computer Engineering, Aarhus University, Denmark*

Abstract

Research on limit order book markets has been rapidly growing and nowadays high-frequency full order book data is widely available for researcher and practitioners. However, it is common that research papers use the best level data only, which motivates us to ask is if the exclusion of the quotes deeper in the book over multiple price levels cause performance degradation. In this paper, we address this question by using modern Machine Learning (ML) techniques to predict mid-price movements without assuming that limit order book markets represent a linear system. We provide a number of results that are robust across ML prediction models, feature selection algorithms, data sets, and prediction horizons. We find that the best bid and ask levels are systematically identified not only to be the most informative levels in the order books, but to carry most of the information needed for good prediction performance. On the other hand, even if the top-of-the-book levels contain most of the relevant information, to maximize models' performance one should use all data across all the levels. Additionally, the informativeness of the order book levels clearly decreases from the first to the fourth level while the rest of the levels are approximately equally important.

Keywords: Limit Order Book, Machine Learning, Feature Selection, Mid-price Movement Prediction

*Corresponding author: Tel.: +358 401592373

Email address: `thanh.tran@tuni.fi` (Dat Thanh Tran¹)

1. Introduction

The modern stock markets are mainly operated by the electronic limit order book (LOB) mechanism, which allows traders and algorithms to immediately use order-book data beyond the best levels for their trading decisions. Intuitively, the contents of a LOB can be informative and useful for trading purposes for many reasons. The order-book asymmetry may reflect trader sentiment or the presence of well informed traders, for which reason one might want to trade in front of its heavy side [1]. Moreover, data beyond the best levels reflects the order-book liquidity, i.e. the quantity immediately available for trading and therefore the price of immediacy. Available liquidity beyond the best levels matters especially when investors' impatience generates a sudden liquidity demand across multiple price levels, leading to large movements in stock prices [2, 3].

Even if the data beyond the best levels were informative, there can be many reasons to use the best-level data only, often referred to as Level-I data, to answer certain research questions. First, parsimonious and tractable models that accomplishes a desired level of explanation or prediction with as few predictor variables as possible can be preferred for the sake interpretability. If models are large and complex, the parameter estimates can be shaky (having large standard errors), leading to statistical insignificance and thus uncertain conclusions. The other reasons to use Level-I data is that data are more easily obtainable than Level-II data that include quotes deeper in the book over multiple price levels. Particularly, Level-I data are easier to pre-process and analyse, and they are well available for academic research. However, when it comes to the maximization of model's predictive power, the current literature lacks to answer the question how informative the top quotes are compared the quotes beyond them. Moreover, it is unclear if one should use multi-level order book data, even at the cost of complexity of the model.

Seeking answers to the research questions above is important in many ways. First, the major portion of the extant financial and econometric literature rely on Level-I data and for different research questions, and it is important to understand how large impact the exclusion of the data beyond the best level can have on the results. The literature uses Level-I data, for example, to analyse micro-structure noise [4, 5], price impact [6, 7, 8, 9], optimal trading strategies [10], and algorithmic trading [11, 12], model and predict the price or order-book dynamics [13, 14]. Even if this paper focuses

on the last topic, i.e. the modelling and prediction of the stock prices with order book data, we believe that our results shed light on other topics, too. In particular, we are efficiently using and assessing on informativeness of the empirical LOB data in terms of the price formation, which is related to trading strategies as well as the use of algorithmic trading. If the inclusion of multi-level order book data did not significantly improve the empirical performance of a prediction model, then it would be sufficient to rely on Level-I data only, which would validate the methodological and data choices of these papers relying on Level-I data.

Second, there is a branch of literature that uses multi-level order-book data to analyze the use of the limit versus market orders [15, 16], order book liquidity [17, 2, 3], market impact [18, 19], and price formation [20]. Recently, the use of complete order-book data has become popular in predicting order-book dynamics or price movements with the advanced machine learning techniques, particularly in the quantitative finance and machine learning literature (see, for example [21, 22, 23, 24, 25, 26, 27, 28] and references therein). Moreover, recently Deep Reinforcement Learning has used for high-frequency trading with LOB data [29]. Even if modern deep-learning techniques allow a huge number of input variables (as long as the size of the data is large), the use of irrelevant input variables is not only unnecessary but can reduce the robustness of the models. Irrelevant data, which potentially acts as a source of noise, can adversely affect the accuracy of the model that captures the underlying relationship between the variables. Moreover, the extra computational burden introduced by additional information can be significant, which renders the analysis impractical for high throughput applications. For these reason, it is important to know if the use of complete Level-II order book data does maximize the predictive power of machine learning models. If this is the case, then one could argue that it is better to use data-driven machine learning techniques with all the available data than parsimonious and tractable models that rely on the top level quotes only.

The question about the importance of the order book data beyond the best level has been barely addressed in the literature, indeed. While there are existing papers that have considered the question about informational contents of LOBs, such as [30, 31], they rely on such simple linear models. In reality, the way how investors and algorithms use order-book data for their trading decisions can be very complicated and vary over different circumstances. If one estimates a specific model assuming pre-specified (linear) relations between the explanatory and dependent variables, the drawn

conclusions can be misleading because of the possibility of model misspecification. More generally, in variable selection (feature selection), it is crucial not to restrict the model to be linear, and in fact, to have any restriction that can incorrectly falsify the importance of a variable. For that reason, the assumption of linearity is not acceptable when it comes to truly non-linear systems, what stock markets definitely represent [32].

In this paper, instead of relying on the linear model paradigm, we aim to address the question about the informativeness of the order book data beyond the best levels by allowing the involved variables to interact in a complex, nonlinear manner. To our best knowledge, this is the first paper that addresses the research question about how informative the order-book is beyond the best levels by a data-driven approach in which the necessity of each explanatory variable and their complex, nonlinear interaction (which also encapsulates the linear case) are jointly estimated. Particularly, we employ state-of-the-arts modeling tools from the machine learning community in our method, namely deep neural networks. It has been shown that feed-forward neural networks with bounded activation functions or rectified linear units are universal function approximators [33]. Simply put, a universal function approximator is guaranteed to have the capacity to approximate arbitrarily well any function that generates the observed data. Thus, from the theoretical perspective, deep neural networks are suitable tools to model the underlying complex relationship between the limit order information and any dependent targets since the only assumption required is the existence of such a relationship. Besides, the most prominent empirical successes over the last few years in financial applications such as stock prediction [34, 35, 36, 37], portfolio selection and optimization [38, 39], factor and risk analysis [40, 41], derivatives hedging [42, 43] and so on, are mainly driven by deep neural network solutions.

In order to discover the importance of different explanatory variables with respect to the dependent variables of interest, our work adapts and integrates well-studied automatic feature selection methods into the training procedure of deep neural networks. By doing so, we are able to gauge the contribution of explanatory variables to the final forecasting decision in a complete data-driven manner. As the name implied, feature selection [44] aims to select the most relevant subset of the explanatory variables to achieve similar or even better learning performance than using all explanatory variables. By developing feature selection methods, not only are we able to identify the most relevant information from the LOB but also to reduce

the computational inference cost when deploying the estimated models in production.

There are three main categories of feature selection methods: filter-based methods, embedded methods, and wrapper methods. Filter-based methods perform selection based on certain criteria measured on the observed data, prior to the estimation of the learning model. Popular criteria used in filter-based methods include reliefF, F-statistic, mRMR or information gain [44]. Because the selection step is decoupled from the model estimation step, the result obtained from a filter-based method is independent of the learning model being used. On the contrary, the feature selection scheme in wrapper-based and embedded methods is intertwined with the learning model, leading to results that are dependent on the models being used. In other words, we might obtain two different subsets of features when applying the same selection scheme to two different learning models.

The machine learning literature mainly concerns about learning models and the final performances. At the same time, we are not aware of any prior work that investigates the consistency among the solutions obtained from a feature selection method applied to different backbone estimation models or even across the solutions obtained from different feature selection methods. Because our main focus is on the informational content of the LOB, interpretations of the outcome are only valid when there is a consensus among different feature selection methods using different backbone estimation models, estimated with observed data from different markets. For this reason, in our work, we adapt and integrate two popular feature selection methods, namely *Backward Elimination* (BE) [45, 46] and *Binary Particle Swarm Optimization* (BPSO) [47], into two state-of-the-arts neural network models that are specifically designed for LOBs, namely *Deep Convolutional Neural Network for LOB* (DeepLOB) [25] and *Temporal Attention Bilinear Layer Network* (TABL) [22].

The idea of BE is that it gradually identifies and removes the most irrelevant features, which, in the context of this paper, are the LOB levels. Particularly, we implemented the algorithm in a way that a given LOB level is removed from the both side of the book.¹ This elimination process continues until there was only one level remaining. By that way, we can analyze how likely a given LOB level remains 'alive' once the algorithm has elimi-

¹In our notation, Level n denotes the n th level on the *both sides* of the book.

nated the others. Because the algorithm removes LOB levels in a way that the performance degradation is minimal, the remaining LOB level can be thought to be the most informative one. On the other hand, with BPSO, an optimal set of input variables is selected such that the fitness function of the neural network model is maximized. This allows us to analyze the appearance of each LOB level in the selected subsets. One can argue that the more often a given LOB level appears in the optimal set of the levels, the more informative the level is. Therefore, both of the approaches show the importance of different LOB levels in the price movement prediction, but in different ways: BE by elimination of the least important levels and BPSO by optimal selection of the sets of the most important levels.

Our empirical analysis was conducted using data coming from two different markets (US and Nordic), which differ in terms the number of order book messages in a given time interval (and thus order book liquidity). Moreover, we vary the length of the prediction horizon from 10 to 30 LOB events. With extensive experimentation, we find out that there is indeed a consensus between different combinations of (i) neural network models, (ii) feature discovery methods, and (iii) markets: the top level of the LOBs provides the most important source of information in predicting the future movements of the mid-price. In addition, there is also a consensus in the ranking between the top three levels of the order-book in terms of importance: the top three levels are also ranked in the same order as their execution priority in the order-book. Our analysis also points out that orders beyond the best level indeed provide complementary information in the prediction of mid-price movements, accounting for 2 to 3 percents of performance improvements compared to the cases where only the most important quotes are used.

To summarize, our paper is organized as follows: Section 2 describes the design and optimization of two state-of-the-arts neural networks that have been proposed for LOBs. In Section 3, we describe our methodology for analyzing the informational content of the LOB. Section 4 provides empirical results and our discussion on the findings. Finally, concluding remarks are drawn in Section 5.

2. Neural Networks for Limit Order Book

Neural networks are a parametric family of mathematical functions that are composed in a hierarchical manner. The abstraction in neural networks is often expressed via layers of processing units. As a high level view, a neural

network consists of the input layer, one or multiple intermediate layers, also referred to as hidden layers, and the output layer. The input layer receives the input data, which can be in the form of a vector or a tensor. The intermediate layers sequentially transform the input data and give the results in the output layer. Each layer in a neural network (except the input layer) represents a mathematical function, which is often parameterized by a set of parameters, also referred to as weights. In a simple feed-forward construction, the k -th layer processes the output of the previous layer, i.e., the $(k-1)$ -th layer. Let us denote the mathematical function expressed by the k -th layer as $\mathcal{G}_k$ and its parameters as Θ_k . The transformation of the k -th layer is described by

$$x_k = \mathcal{G}_k(\Theta_k, x_{k-1}), \quad (1)$$

where x_k denotes the output of the k -th layer. Depending on the types of neurons forming the neural network, Θ_k can be a matrix as in fully-connected layers or Θ_k can be a tensor as in convolutional layers. While the form of $\mathcal{G}_k$ is often determined by human experts via domain knowledge and experimentation, most common layer designs such as the fully-connected layer and the convolution layer typically involve some linear transformations of the input via matrix multiplication followed by element-wise nonlinear activation. In addition, a single choice of nonlinear activation function, such as the Rectified Linear Unit (ReLU), is often applied to all layers to narrow down the possible choices when designing a neural network solution, except for the output layer, which has an activation function depending on the given task. For example, the soft-max function, which transforms an un-normalized vector to a vector having positive elements summing up to one, thus, representing the class probability prediction, is a de-facto choice for classification tasks.

Here we should note that in many state-of-the-arts neural network formulations, the connectivity patterns can be more complicated than the one described in Eq. (1) for a simple feed-forward construction. That is, a layer can not only take the output of its immediate preceding layer, but also the output of other preceding layers. After defining the architecture of a neural network model, i.e., the types of the input data, the number of layers, the type of transformation of each layer and their connectivity pattern, its parameters are estimated by optimizing a learning objective (measured by a loss function) using the set of (training) data. That is, the network's parameters

$(\Theta = \{\Theta_1, \dots, \Theta_K\})$ are determined by solving

$$\operatorname{argmin}_{\Theta} \sum_{i=1}^N L(y_i, \tilde{y}_i), \quad (2)$$

where L denotes the loss function, y_i denotes the true label of the i -th sample and $\tilde{y}_i$ denotes the corresponding prediction by the neural network.

Since the objective in Eq. (2) is highly nonlinear for neural networks with multiple layers, a variant of the gradient descent algorithm is often used to find Θ . As the name implies, gradient descent is an iterative algorithm that updates the parameters of a function by repeatedly moving the current parameters' values along the steepest slope (the gradient). In its simplistic form, a gradient descent algorithm has the following update rule:

$$\Theta(t+1) = \Theta(t) - \alpha \frac{\partial L}{\partial \Theta(t)}, \quad (3)$$

where $\Theta(t)$ denotes the value of Θ at step t , and α is a scalar for controlling the amount of the update. The value of α can be sensitive to the convergence of the gradient descent algorithm since a large value can easily lead to divergence while a small value makes it slow to converge. There have been many extensions to the basic gradient descent algorithm described in Eq. (3). Among those, SGD with momentum [48] and ADAM [49] are the most popular ones in the neural network community.

In the following, we will describe two state-of-the-arts neural network formulations for the mid-price movement prediction task, namely **TABL** and **DeepLOB**. Both networks take as input the most recent states of the order-book, which include the top ten quotes (price and volume) from the bid and the ask sides. For a given stock at time t , let us denote the top- k bid and ask prices as $p_k^b(t)$ and $p_k^a(t)$, and volumes as $v_k^b(t)$ and $v_k^a(t)$. The information of the top ten levels at each time instance is thus represented by a 40-dimensional vector. Since **TABL** and **DeepLOB** take advantage of both the current and past order-book information, the input data is a multivariate time-series having dimension of $40 \times T$ with T being a hyperparameter that determines the length of past information.

2.1. Temporal Attention Augmented Bilinear Layer Networks

Temporal Attention augmented Bilinear Layer network (**TABL**) was formulated by [22] to specifically work with financial multivariate time-series.

There are two main ideas in the design of TABL networks: (i) the utilization of bilinear layers to capture linear dependency along the feature and temporal dimension of the input and intermediate series, and (ii) the incorporation of an attention mechanism to automatically identify and focus on important temporal instances within a series.

A bilinear layer (BL) takes as input a matrix $\mathbf{X} \in \mathbb{R}^{D \times T}$ and produces a matrix $\mathbf{Y} \in \mathbb{R}^{D' \times T'}$ via transformation

$$\mathbf{Y} = \phi(\mathbf{W}_1 \mathbf{X} \mathbf{W}_2 + \mathbf{B}), \quad (4)$$

where $\mathbf{W}_1 \in \mathbb{R}^{D' \times D}$, $\mathbf{W}_2 \in \mathbb{R}^{T \times T'}$, and $\mathbf{B} \in \mathbb{R}^{D' \times T'}$ denote the parameters associated with the bilinear layer. ϕ denotes an element-wise activation function such as ReLU or sigmoid. In short, a BL layer linearly transforms the input time-series $\mathbf{X}$ along the first (the feature) and second (the temporal) dimension separately by using $\mathbf{W}_1$ and $\mathbf{W}_2$, then shifts the intermediate result with the bias term $\mathbf{B}$ before the nonlinear thresholding operation.

The benefit of using a bilinear transformation for multivariate time-series is two-fold. The first advantage is the reduction of parameters that need to be estimated, compared to a linear transformation. This is because a linear mapping, which transforms a vector from $D \cdot T$ to $D' \cdot T'$ dimensions, requires the estimation of a transformation matrix having $D \cdot T \cdot D' \cdot T'$ elements while the bilinear transformation in Eq. (4) using $\mathbf{W}_1$ and $\mathbf{W}_2$ only requires the estimation of $D \cdot D' + T \cdot T'$ elements. The second advantage is the preservation of semantic structure within input time-series. In Eq. (4) since $\mathbf{W}_1$ linearly combines the rows of $\mathbf{X}$ and $\mathbf{W}_2$ linearly combines the columns of $\mathbf{X}$, a BL layer effectively captures the linear dependency between different univariate series within $\mathbf{X}$ via $\mathbf{W}_1$, and the linear dependency between different temporal observations via $\mathbf{W}_2$ (note that the second dimension of $\mathbf{X}$ represents the temporal dimension). On the hand, using a linear mapping requires the vectorization of $\mathbf{X}$, thus breaking the structural information inherent in the input series.

TABL architecture follows a simple feed-forward connectivity pattern in which one or a few BL layers are stacked one after another in a sequential manner to form the hidden layers. The output layer of a TABL network is TABL layer, which extends the standard BL layer with an attention mechanism on the temporal dimension. Given the same input and output notation as in a BL layer, the transformation performed by a TABL layer consists of five steps:

- (i) The input series $\mathbf{X}$ is first linearly transformed along the first dimension, i.e., the feature dimension, using $\mathbf{W}_1$:

$$\bar{\mathbf{X}} = \mathbf{W}_1 \mathbf{X}. \quad (5)$$

- (ii) The relative importance between different temporal instances (columns) in the intermediate feature matrix $\bar{\mathbf{X}}$ is computed via a specially designed matrix $\mathbf{W}$,

$$\mathbf{E} = \bar{\mathbf{X}} \mathbf{W}, \quad (6)$$

where $\mathbf{W} \in \mathbb{R}^{T \times T}$ is a parameter matrix that has constant values $(1/T)$ on its diagonal. Other elements of $\mathbf{W}$ are estimated during optimization to determine the pair-wise relevance scores between columns of $\bar{\mathbf{X}}$.

- (iii) The relevance score at each position $[i, j]$ of $\mathbf{E}$ is then normalized over the second dimension (the temporal dimension) using the soft-max function to produce an attention mask $\mathbf{A} \in \mathbb{R}^{D' \times T}$, with the element at position $[i, j]$ computed as

$$\mathbf{A}[i, j] = \frac{\exp(\mathbf{E}[i, j])}{\sum_{k=1}^T \exp(\mathbf{E}[i, k])}. \quad (7)$$

Since $\mathbf{E}$ is normalized along the second dimension, elements within each row of the attention mask $\mathbf{A}$ sums up to one. In this way, the attention mask encodes the relative importance of a temporal slice (a column) with respect to others.

- (iv) The attention matrix $\mathbf{A}$ is then used to mask out irrelevant elements in the intermediate feature $\bar{\mathbf{X}}$. This is done via a *soft* attention mechanism, which performs convex combination of $\bar{\mathbf{X}}$ and its masked-out version:

$$\tilde{\mathbf{X}} = \lambda(\bar{\mathbf{X}} \odot \mathbf{A}) + (1 - \lambda)\bar{\mathbf{X}}. \quad (8)$$

Here λ is a scalar, which is constrained to have a value between $[0, 1]$ and is also estimated during the optimization procedure.

- (v) In the final computation step, the attended feature matrix $\tilde{\mathbf{X}}$ is linearly transformed in the second dimension with $\mathbf{W}_2$, and shifted by the bias term $\mathbf{B}$, before going through the activation function ϕ to produce the output of the layer:

$$\mathbf{Y} = \phi(\tilde{\mathbf{X}} \mathbf{W}_2 + \mathbf{B}) \quad (9)$$

Compared with the standard BL layer, the TABL layer requires an additional estimation of $\mathbf{W}$ and λ for the computation of the attention mechanism. Since both BL and TABL layers are differentiable, gradient descent updates can be easily computed. In the original paper, the authors used ADAM [49], a stochastic gradient descent optimizer that performs adaptive estimate of the update size using lower-order moments. In this work, we also use ADAM with the same configuration as in [22] to optimize TABL networks. Regarding the architecture, [22] evaluated three different TABL networks with the number of hidden layers ranging from zero to two. Despite being shallower and having far fewer parameters, a TABL network with two hidden layers produces superior prediction performance compared to existing neural network solutions, both shallow and deep nets. For this reason, we use the same two-hidden-layer network topology proposed by [22]: this network takes an input series of size 40×10 , containing the quote data of the ten most recent order-book states; two hidden layers are BL layers, which have the output dimensions of 60×10 and 120×5 , respectively; the output layer is a TABL layer, which has the output dimension of 3×1 , encoding the probability of three types of future mid-price movements.

2.2. Deep Convolutional Neural Networks for Limit Order Books

Beside the TABL network from [22], the deep neural network design called DeepLOB proposed by [25] has also demonstrated excellent performances in predicting mid-price movements using LOB information. Different from TABL, DeepLOB is a deep network with several convolution layers. In addition, DeepLOB also stacks a recurrent layer after the convolution layers and before the output layer, which is a fully-connected layer that performs linear mapping and soft-max activation to produce a probability prediction vector.

As the name implies, a convolution layer applies convolutional transformations of the input via a set of convolution filters. Given an input matrix $\mathbf{I} \in \mathbb{R}^{H \times W}$, a convolution filter $\mathbf{w} \in \mathbb{R}^{h \times w}$, and a stride length of $s_1 \times s_2$, the result of convolving $\mathbf{X}$ with $\mathbf{w}$ using the given stride length is a matrix $\mathbf{Y}$, with the element at position $[p, q]$ computed as

$$\mathbf{Y}[p, q] = \sum_{i=1}^h \sum_{j=1}^w \mathbf{w}[i, j] \cdot \mathbf{I}[i + (p-1) \cdot s_1, j + (q-1) \cdot s_2], \quad (10)$$

where $\mathbf{Y}$ has dimensions of $H' \times W'$, with $H' = \lfloor (H - h)/s_1 + 1 \rfloor$ and $W' = \lfloor (W - w)/s_2 + 1 \rfloor$.

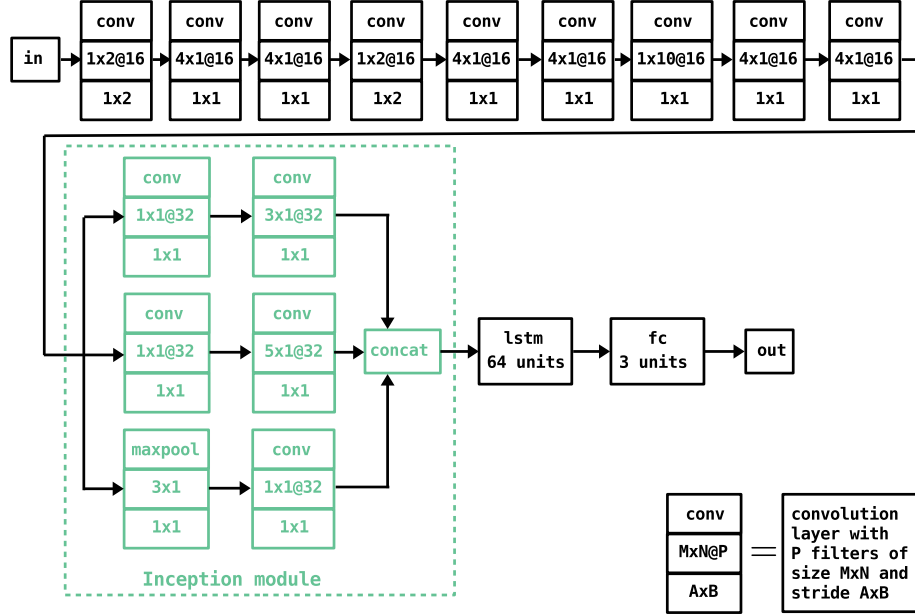


Figure 1: DeepLOB architecture from [25]

In order to extract low level features from the input time-series, DeepLOB utilizes a series of nine convolution layers with different configurations, which are depicted in Figure 1. The output of the last convolution layer is fed to a computation module called Inception. The design of the Inception module in DeepLOB was inspired from the work of [50], in which the authors proposed a very efficient neural network architecture, called Inception, for solving computer vision problems. The key idea behind the Inception design is the utilization of convolution layers with different filter sizes in parallel to extract multi-scale features. That is, instead of applying one convolution layer after the output of another, the Inception module contains three parallel branches of convolution layers that are applied to the same input, and concatenates the output of all branches to form the output of the module. The Inception design is illustrated in Figure 1.

After extracting multi-scale features using the Inception module, DeepLOB further processes multi-scale features using a Long-Short-Term-Memory (LSTM) layer [51]. LSTM is a type of recurrent layer, which was designed specifically to extract fixed-length representation from variable-length sequences such as those encountered in natural language processing problems. The ability to

transform variable-length sequences to fixed-length sequences comes from the fact that a recurrent layer applies recursive computation to each temporal slice of the input sequence using both the current temporal slice and the outputs of the previous computation as inputs. Mathematically, let us denote the input sequence as $\mathbf{X} \in \mathbb{R}^{D \times T}$ with the second dimension representing the temporal dimension. In addition, $\mathbf{x}_t \in \mathbb{R}^D$ denotes the t -th temporal slice (column) of $\mathbf{X}$. The recursive computation of a recurrent layer can be described by

$$\mathbf{h}_t, \mathbf{o}_t = \mathcal{F}(\mathbf{h}_{t-1}, \mathbf{x}_t), \forall t = 1, \dots, T, \quad (11)$$

where $\mathcal{F}$ denotes some function that takes two vectors, namely $\mathbf{h}_{t-1}$ and $\mathbf{x}_t$ as inputs, and also returns two vectors $\mathbf{h}_t$ and $\mathbf{o}_t$ as outputs.

In recurrent network terminology, $\mathbf{h}_t \in \mathbb{R}^{D_h}$ is referred to as the hidden state at time t and $\mathbf{o}_t \in \mathbb{R}^{D_o}$ is known as the cell state or intermediate output at time t . The last cell state, i.e., $\mathbf{o}_T$ is also the output of the recurrent layer. Here D_h and D_o are the dimensions of the hidden states and the outputs, which are typically set similar, and often referred to as the number of units in a recurrent layer. The specific transformations in $\mathcal{F}$ depend on the design of the recurrent layer. There are many different recurrent layer designs, among which LSTM and GRU [52] are the most widely used. The output layer of DeepLOB performs a linear transformation on the output of the LSTM layer, then soft-max activation to generate a probability prediction vector.

3. Informational Content Discovery via Feature Selection

As we have mentioned in Introduction, our objective is to study the informational content of the quote data across multiple levels by analyzing the predictive power of each level of quotes. For this purpose, we adopted two feature selection methods to incorporate into the TABL and DeepLOB networks. The general idea of feature selection is to identify a subset of input variables that can effectively capture the most relevant information from the observed data for a given learning task. Since the number of input variables is finite, this is essentially a search problem in a discrete space. Depending on the nature of the data and the main purpose behind having such a subset of features, the criterion for the search problem might be constructed in different ways. For example, one might be interested in finding the smallest subset of features to reduce future computational complexity when working with the given problem. Others might be merely interested in the relevance of each feature and their relative ranking in terms of relevance.

Typically, feature selection is formulated as finding the subset of input features from which the estimated learning model achieves the best performance. A global solution for this criterion requires estimating at least 2^D models, corresponding to all 2^D subsets of features, with D denoting the number of input variables. For the class of models that are estimated with some degree of stochasticity, one would need to estimate far more than 2^D models to ensure statistical significance of the conclusion. Even for a model with closed-form solution such as linear models, a greedy solution becomes exponentially slow as the number of input variables increases. For this reason, the feature selection criteria might be formulated differently [44], such as finding the subset of a given size to achieve the best performance or finding a subset of features that achieve similar or better performance compared to the baseline model that is estimated using the full set of features.

In order to discover the predictive power of different levels in the order-book, we selected two feature selection methods, namely Backward Elimination (BE) [45, 46] and Binary Particle Swarm Optimization (BPSO) [47] with different search trajectories. While BE produces a candidate subset of a given size that may achieve the best performance, the search trajectory of BPSO can travel the entire search space and converges at a subset of any size, which is not predefined. Since our selected classifiers (TABL and DeepLOB) are estimated using stochastic gradient descent, it is possible that different runs of either BE or BPSO can generate different subsets. Thus, to analyse the informational content of our inputs, we look for any consensus among the solutions provided by both methods applied to the selected classifiers, using two different markets' data. Our approach to look for such consensus is elaborated in Section 4.

The idea of BE is very simple: the algorithm starts with the set of all input features, then gradually identifies and removes the most irrelevant feature, one at a time, until there is only a single level left or the learning performance falls below a pre-defined threshold. The majority of existing literature that uses BE employed simple linear models, thus F -test or t -test are popular criteria to identify the irrelevant feature at each elimination step. For the approach that utilizes a linear regressor, we only need to perform one estimation of the model's parameters and use them to identify which feature to remove. In our case where highly nonlinear neural networks are employed, there is no straight-forward method to quantify the importance of a given input variable based on the optimized weights. For this reason, in order to determine data from which order-book level should be discarded in

an elimination round having K remaining levels, we train K different neural network instances, each of which corresponds to leaving out the quotes of a potentially irrelevant level. The irrelevant level is then identified with the network instance that achieves the lowest prediction performance. In our experiments, we performed the elimination routine until there was only one level left. Here we should point out that since these K neural network instances are independent of each other, they can be optimized in parallel in practice.

One might be concerned with our selection of the BE algorithm since in some prior research employing BE to study the explanatory or predictive power of input variables in modeling tasks, the algorithm exhibited certain degrees of instability with respect to small perturbation of data or noise, which was considered as having no effect on the dependent variable [53, 54, 55]. Here we should note that the nature of our problem and data is different from those scenarios for two reasons: (i) the data coming from electronic LOBs can be considered as rather *clean* in the sense that it is expected to contain very little to no noise such as those coming from an imperfect measurement/collection process in other application areas. This is especially true from a technical point of view for modern, completely electronic systems in big exchanges, which are thoroughly designed, tested and equipped with backups. Thus, the simulation of noise by introducing small perturbation to the data does not represent a real scenario. (ii) Small perturbation of the quote price cannot be considered as insignificant or having no effect to the dependent variables.

While BE resembles more of a series of heuristic tests to identify important features, BPSO is a generic, well-studied meta-heuristic optimizer for binary variables, rather than a dedicated feature selection method. That is, BPSO optimizes a binary vector such that a fitness function computed on it is maximized. In order to use BPSO to perform feature selection, one would define a binary mask vector having the same dimension as the number of features, which represents the selection of a particular subset of features. The values of this binary mask vector are randomly initialized and iteratively updated until convergence. BPSO applied in such manner considers each input variable independently while in our case, we want to form subset of the quote values that correspond to specific order-book levels. This necessitates modifications as how BPSO is formulated for LOBs.

Let us recall from Section 2 that our input variables are the most recent order-book events, which include the top ten bid prices $(p_1^b(t), \dots, p_{10}^b(t))$

and volumes $(v_1^b(t), \dots, v_{10}^b(t))$, and top ten ask prices $(p_1^a(t), \dots, p_{10}^a(t))$ and volumes $(v_1^a(t), \dots, v_{10}^a(t))$, with t indicating the time index. These variables are organized in the following matrix to reflect a multivariate time-series nature of the order-book events:

$$\mathbf{X}_t = \begin{bmatrix} p_1^a(t-T-1) & \dots & p_1^a(t) \\ v_1^a(t-T-1) & \dots & v_1^a(t) \\ p_1^b(t-T-1) & \dots & p_1^b(t) \\ v_1^b(t-T-1) & \dots & v_1^b(t) \\ \vdots & \dots & \vdots \\ p_{10}^a(t-T-1) & \dots & p_{10}^a(t) \\ v_{10}^a(t-T-1) & \dots & v_{10}^a(t) \\ p_{10}^b(t-T-1) & \dots & p_{10}^b(t) \\ v_{10}^b(t-T-1) & \dots & v_{10}^b(t) \end{bmatrix} \in \mathbb{R}^{40 \times T}, \quad (12)$$

where T denotes the number of the most recent order-book events that a neural network predictor uses as inputs. In our experiments, we used ten most recent order events, i.e., $T = 10$. In addition, each sample $\mathbf{X}_t$ comes with a label y_t that indicates the future mid-price movement.

Let us denote by $\mathbf{s} \in \mathbb{R}^{10}$ the binary vector that indicates a particular subset of ten levels, with $\mathbf{s}[i] = 1$ indicating the inclusion of level i and $\mathbf{s}[i] = 0$ otherwise. In addition, we also denote by $\mathbf{M}(\mathbf{s}) \in \mathbb{R}^{40 \times T}$ a binary matrix that is formed from $\mathbf{s}$ as

$$\mathbf{M}(\mathbf{s})[i, j] = \mathbf{s}[k], \quad \forall j = 1, \dots, T \quad (13)$$

$$k = \lfloor (i-1)/4 \rfloor + 1, \quad \forall i = 1, \dots, 40. \quad (14)$$

That is, $\mathbf{M}(\mathbf{s})$ denotes the binary matrix that can be used to mask out all the values in $\mathbf{X}_t$ that belongs to unchosen levels expressed in $\mathbf{s}$. Here we should note that $\mathbf{M}(\mathbf{s})$ can also be defined via a matrix multiplication operation as $\mathbf{M}(\mathbf{s}) = \mathbf{I}_1 \mathbf{s} \mathbf{I}_2$ with $\mathbf{I}_1 \in \mathbb{R}^{40 \times 10}$ and $\mathbf{I}_2 \in \mathbb{R}^{1 \times T}$ being two constant matrices with appropriate ones and zeros. Now the optimization objective of BPSO can be stated as

$$\underset{\mathbf{s}}{\operatorname{argmax}} \quad \mathcal{F}_1(\{y_t, \mathcal{N}(\mathbf{X}_t \odot \mathbf{M}(\mathbf{s})) | t = 1, \dots\}), \quad (15)$$

where $\mathcal{N}$ denotes the function representing the neural network giving predictions and Θ is its parameters. $\mathcal{F}_1$ denotes the function that computes average F1 measure in a classification problem given the set of true labels

and predicted labels. The element-wise multiplication operation at the end of Eq. (15) is denoted as $\odot$.

Here we should note that the F1 measure is used in our analysis because it reflects the trade-off between precision and recall, which is especially important for problems with skewed distributions as observed in our data.

The idea in **BPSO** is that we may discover an optimal solution faster by using a list of candidate solutions, which are also called particles. Let us denote the k -th particle in the swarm as $\mathbf{s}_k$ and the size of the swarm as K . The swarm of particles moves together in the search space to find the optimal solution, thus, each of them has a velocity, which is denoted as $\mathbf{v}_k \in \mathbb{R}^{10}$. At each movement, a particle moves to a new position with its velocity, which is computed based on its past experience and also the experiences of other particles. Let us denote the best position that the k -th particle has visited as $\mathbf{s}_k^*$, and the best position that the entire swarm has visited as $\mathbf{s}^*$. The k -th particle adjusts its velocity with the following rule:

$$\mathbf{v}_k^{(i+1)} = w \odot \mathbf{v}_k^{(i)} + c_1 \cdot r_1 \cdot (\mathbf{s}_k^* - \mathbf{s}_k^{(i)}) + c_2 \cdot r_2 \cdot (\mathbf{s}^* - \mathbf{s}_k^{(i)}), \quad (16)$$

where i represents the iteration index in the optimization routine. w is the inertia term, which is a hyperparameter of the method to control the impact of the old velocity on the new one. The value of w is often reduced gradually as the swarm progresses. $c_1 \in \mathbb{R}$ and $c_2 \in \mathbb{R}$ are acceleration constants and r_1 and r_2 are randomly sampled scalars from the uniform distribution $[0, 1]$. To control the maximum velocity of a particle at each iteration, the velocity is limited within a range $[-\mathbf{v}_{\max}, \mathbf{v}_{\max}]$.

Since a particle can only move in a discrete space of 0 and 1, the velocity is used to determine the probability that a particle will move to position 0 or 1. With the new velocity $\mathbf{v}_k^{(i+1)}$, the k -th particle updates its position using the following rule:

$$\mathbf{s}_k^{(i+1)}[j] = \begin{cases} 1 & \text{if } r_3 < \text{sigmoid}(\mathbf{v}_k^{(i+1)}[j]) \\ 0 & \text{else,} \end{cases} \quad (17)$$

where $\mathbf{s}_k^{(i+1)}[j]$ and $\mathbf{v}_k^{(i+1)}[j]$ denote the j -th element in $\mathbf{s}_k^{(i+1)}$ and $\mathbf{v}_k^{(i+1)}$, respectively. r_3 is a randomly sampled scalar from the uniform distribution $[0, 1]$.

Given the new positions of the swarm, **BPSO** evaluates the fitness of each particle by optimizing the neural network's parameters, i.e., for the k -th

particle, we need to solve the following optimization problem with stochastic gradient descent:

$$\Theta^* = \operatorname{argmin}_{\Theta} \sum_t L \left(y_t, \mathcal{N} \left(\mathbf{X}_t \odot \mathbf{M} \left(\mathbf{s}_k^{(i)} \right) \right) \right), \quad (18)$$

where Θ is the parameters of the neural network $\mathcal{N}$, and Θ^* denotes its optimal values obtained from stochastic gradient descent. L denotes the cross-entropy loss function.

The fitness for the k -th particle at the i -th iteration is defined as the F1 score obtained at Θ^* . Based on new fitness values for all particles, BPSO checks and updates $\mathbf{s}_k^*$ and $\mathbf{s}^*$ if new best solutions emerge. The algorithm repeats the aforementioned procedure for a pre-defined number of iterations.

4. Empirical Results

4.1. Data Collection and Processing

We analysed the informational content of the LOB data coming from two different markets: the Nordic market and the US market. For the Nordic market, we collected orders from five companies that are listed on Helsinki Stock Exchange, namely Kesko, Outokumpu, Rautaruukki, Sampo and Wartsila, and for the US market, we used orders coming from Amazon and Google. All of our data is procured from TotalView-ITCH feed with the resolution in milliseconds, covering a period of ten working days from the 22nd of September 2015 to the 5th of October 2015. Data from the message books was processed to form the LOBs of depth ten for both bid and ask sides.

For the Helsinki Stock Exchange, the trading period goes from 10:00 to 18:25 (local time, UTC/GMT+2) while for the Nasdaq US Stock Market, the trading hour spans from 09:30 to 16:00 (EST). For Nordic stocks, we excluded the auction period and only considered data between 10:30 and 18:00 following the previous convention used for this data [2, 3]. For the US stocks, we considered all trading hours. This resulted in approximately 13 millions events for the US dataset and 4 millions events for the Nordic dataset, with the duration between consecutive events unevenly distributed, ranging from milliseconds to minutes.

The data of the first seven days was used to train and validate the models and the data covering the last three days was used as the held-out test set. For each market, we aggregated the data from all stocks and used

them to train stock-agnostic models to predict the future movement of the mid-price after $H = \{10, 20, 50\}$ order events. The future movements of the mid-price belongs to one of the three categories: up, down, and stationary. Since financial data often exhibits stochasticity, simply deriving labels for mid-price movements by comparing the current mid-price with the one in the future will lead to a very noisy label set. To reduce this effect, previous practices have employed price smoothing for the labeling process [56, 25], and we followed the same procedure that smoothes the mid-price using both future and past values as follows:

$$y_t \equiv \begin{cases} \text{up,} & l_t > \alpha \\ \text{stationary,} & -\alpha \leq l_t \leq \alpha \\ \text{down,} & l_t < -\alpha, \end{cases} \quad (19)$$

where $\alpha = 0.00001$ is a constant to define the smallest margin change to be considered as a movement. The quantity l_t is defined as

$$l_t = \frac{m_+(t) - m_-(t)}{m_-(t)} \quad (20)$$

$$m_-(t) = \frac{1}{H} \sum_{i=0}^H p(t-i) \quad (21)$$

$$m_+(t) = \frac{1}{H} \sum_{i=1}^H p(t+i). \quad (22)$$

For machine learning models in general and for neural networks in particular, data normalization can heavily affect the final performance. For high-frequency trading data, prior works [28, 57] have shown that a suitable data normalization method can significantly boost the performance of the same model. In our case, since we aimed to train stock-agnostic model using their order information, the price range and the volume can be significantly different for different stocks, thus, requiring attention on data normalization scheme. Z-score normalization, i.e., subtracting the mean from the data and dividing the values with standard deviation, is the most common scheme. For a large amount of time-series data spanning several days, it is also a common practice to perform Z-score normalization with statistics coming from the previous day, rather than from the whole training set spanning several days [25]. There are also other advanced data normalization methods

that estimate the normalization parameters in conjunction with a learning model's parameters [58, 59, 28, 57]. Based on the results of our preliminary experiments with Z-score normalization and two adaptive methods [28, 57], we adopted the bilinear normalization technique proposed in [57] for our experiments since this method exhibits strong performance over the others.

As a final note on data normalization procedure, depending on the data normalization methods, the normalized data might inherently contain information of all features, thus, can interfere with the judgement of relevant subset of features because the chosen subset of normalized features already contains some information, which is obtained from the discarded features. For this reason, instead of normalizing the whole data once and proceed with feature selection algorithms, we only performed data normalization after the subset selection step. For example, in BE when evaluating K different neural networks using K different subsets of the remaining levels in an elimination round, the train sets corresponding to K subsets are normalized independently, using only statistics of the corresponding subset.

4.2. Hyperparameters and Optimization Details

Regarding the optimization of TABL and DeepLOB, we used ADAM optimizer [49] with $\beta_1 = 0.9$ and $\beta_2 = 0.999$ for both networks in all experiments. To optimize a neural network instance, we randomly initialized the parameters with the initialization technique proposed in [60] and updated the parameters for 50 epochs with the initial learning rate of 0.001, which was reduced by a factor of 10 at epoch 6 and 46. The batch size for all experiments was set to 128 and weight decay regularization coefficient was set to 0.00005. To counter the effect of imbalanced class observed in the mid-price movement labels, when training the networks, we adopted the same weighted cross-entropy loss function proposed in [22], which basically multiplies a weight scalar, which is inversely proportional with the size of a class, to the loss terms associated with that class.

Regarding BPSO, we used a swarm of 15 particles for all experiments. The inertia term w in Eq. (16) was computed using an annealing schedule, with the value of w at the i -th iteration computed as $w = 2 - 2 \cdot (i/I)^{1/\pi^2}$ with $I = 20$ being the maximum number of iterations. In Eq. (16), $c_1 = c_2 = 1.5$, and the velocity was limited within the range $[-6, 6]$. The selected values for hyperparameters of BPSO are conventional choices adopted in literature.

Given two types of prediction networks (TABL and DeepLOB), two feature selection methods (BE and BPSO), two datasets (US and Nordic), and three

prediction horizons ($H = \{10, 20, 50\}$ events), our total experiment set consists of 24 different configurations. For each configuration, we repeated the experiment 20 times and reported the mean and standard deviation values for F1 score measured on the held-out test set.

4.3. Are the top quotes the most important predictors?

Based on the experiment results collected, the first question that we sought an answer is whether the assumption about the top quotes being the most informative information of the LOBs still holds even when using highly nonlinear models for the modeling task. Let us recall from Section 3 that as the elimination goes in BE, we sequentially obtain the most important subsets of LOB levels of decreasing sizes, and also the associated test performance that can be achieved from the selected subset. This process was conducted until there was only one level remaining. If we look into the percentage that a given level appeared as the only remaining level after running BE, out of 20 experiment repetitions for each configuration, we obtained the following statistics for ten levels in Table 1.

Table 1: The percentage that a given level (on the both sides of the book) appeared as the only remaining level when using BE. The results are provided for US and Nordic data sets and for three different prediction horizons, $H = 10, 20, 50$. Moreover, two models for the price prediction with the LOB data are used, **DeepLOB** [25] and **DeepLOB** [22].

H	US Data						Nordic Data					
	DeepLOB			TABL			DeepLOB			TABL		
	10	20	50	10	20	50	10	20	50	10	20	50
Level 1	100%	100%	100%	100%	100%	100%	95%	90%	95%	65%	95%	100%
Level 2	0%	0%	0%	0%	0%	0%	0%	5%	0%	25%	5%	0%
Level 3	0%	0%	0%	0%	0%	0%	0%	0%	0%	5%	0%	0%
Level 4	0%	0%	0%	0%	0%	0%	0%	5%	5%	5%	0%	0%
Level 5	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
Level 6	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
Level 7	0%	0%	0%	0%	0%	0%	5%	0%	0%	0%	0%	0%
Level 8	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
Level 9	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%
Level 10	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%

As it is obvious from Table 1, the first LOB level consistently appeared with high percentages as the last level remaining during the elimination in BE across different configurations. Most notably, for the US data, the results

of all configurations unanimously selected level 1 as the most predictive level. The results for Nordic data also indicated the same phenomenon with level 1 being selected at least 18 out of 20 times (90%) for all configurations, with the only exception when using **TABL** to predict at $H = 10$.

We will now look into the results obtained from **BPSO** from the same aspect. Let us recall from Section 3 that the subset of levels selected by **BPSO** can be of any cardinality. The solution given by **BPSO** at the end of the optimization process is the best solution among all the positions that the swarm visited during its course. To seek for potential consensus between **BPSO** and **BE**, we computed the percentage that a given level appeared in the final solution given by **BPSO**, out of 20 experiment runs for each configuration. The statistics are shown in Table 2.

Table 2: The percentage that a given level (on the both sides of the book) appeared in the final solution of **BPSO**. The results are provided for US and Nordic data sets and for three different prediction horizons, $H = 10, 20, 50$. Moreover, two models for the price prediction with the LOB data are used, **DeepLOB** [25] and **DeepLOB** [22].

H	US Data						Nordic Data					
	DeepLOB			TABL			DeepLOB			TABL		
	10	20	50	10	20	50	10	20	50	10	20	50
Level 1	100%	100%	60%	100%	100%	80%	60%	50%	50%	50%	90%	70%
Level 2	20%	10%	30%	0%	10%	10%	30%	20%	20%	20%	10%	20%
Level 3	0%	10%	10%	10%	0%	20%	10%	30%	10%	30%	0%	0%
Level 4	0%	0%	0%	0%	10%	0%	0%	0%	10%	0%	0%	0%
Level 5	0%	0%	0%	10%	10%	0%	0%	10%	10%	0%	10%	0%
Level 6	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	0%	10%
Level 7	0%	0%	0%	10%	0%	0%	0%	0%	0%	0%	0%	0%
Level 8	0%	0%	10%	0%	0%	0%	0%	0%	0%	0%	0%	0%
Level 9	0%	10%	0%	20%	0%	0%	0%	0%	0%	0%	0%	0%
Level 10	10%	0%	10%	0%	0%	0%	0%	0%	0%	0%	0%	0%

Compared to the statistics obtained from **BE**, we observed more variation in the results when running **BPSO**, with the majority of variations associated with the Nordic data. This phenomenon resembles what appeared for the **BE** experiments in Table 1. Despite the results being less unanimous, we can still observed the fact that level 1 was repeatedly selected by the swarm-based solver with percentages exceeding other levels by large margins. In addition, Table 2 also shows that there is a consensus between the two feature selection methods about level 1 being the most important predictor.

Table 3: The percentage that a given level appeared in the subset containing two levels selected by BE algorithm. The results are provided for US and Nordic data sets and for three different prediction horizons, $H = 10, 20, 50$. Moreover, two models for the price prediction with the LOB data are used, DeepLOB [25] and DeepLOB [22].

H	US Data						Nordic Data					
	DeepLOB			TABL			DeepLOB			TABL		
	10	20	50	10	20	50	10	20	50	10	20	50
Level 1	100%	100%	100%	100%	100%	100%	100%	100%	100%	80%	100%	100%
Level 2	10%	20%	15%	15%	35%	25%	5%	10%	30%	35%	55%	50%
Level 3	0%	20%	25%	20%	20%	15%	5%	5%	10%	10%	5%	20%
Level 4	10%	0%	5%	5%	20%	25%	15%	10%	5%	20%	5%	10%
Level 5	10%	5%	10%	0%	0%	0%	10%	5%	20%	10%	0%	5%
Level 6	5%	15%	5%	20%	10%	15%	5%	15%	5%	10%	5%	10%
Level 7	15%	10%	20%	15%	5%	5%	25%	20%	5%	15%	5%	0%
Level 8	20%	15%	5%	5%	5%	0%	15%	10%	20%	10%	15%	0%
Level 9	25%	15%	0%	15%	0%	0%	10%	20%	0%	5%	5%	0%
Level 10	5%	0%	15%	5%	5%	15%	10%	5%	5%	5%	5%	5%

Table 4: The average percentage of appearance in a 2-element subset selected by BE algorithm over all prediction horizons, neural network models and datasets, i.e., average over all columns in Table 3.

Level 1	Level 2	Level 3	Level 4	Level 5	Level 6	Level 7	Level 8	Level 9	Level 10
98.33%	25.42%	12.92%	10.83%	6.25%	10.00%	11.67%	10.00%	7.92%	6.67%

4.4. What is the second most important predictor?

Given that the quote data from level 1 are the most important source of information in the order-book, which level comes second? In order to seek an answer for this question, we need to look at another statistic from the results. Particularly, from the set of results obtained from BE method, we looked at the percentage that a given level appeared in the subsets selected by BE having a cardinality of 2. Due to the sequential removal nature of BE, we know for certain that the numbers for level 1 are similar or higher than those appear in Table 1. But can we observe a similar phenomenon for any other level when BE is allowed to form a subset of two levels?

The statistics for this inquiry are shown in Table 3. The numbers in this table indicate that there is no level that seems to consistently appear by more than 50% with the top quote data in 2-element subsets chosen by BE. In fact, for each level, we computed the average percentage of appearance in

a 2-element subset selected by BE over all prediction horizons, neural network models and datasets, i.e., the average number for every row in Table 3, we observed that the second highest record comes from level 2, however, at only 25.42%. The records for all levels are shown in Table 4. By following the same procedure and computing the average percentage of each level appearing in the 1-element, 2-element and 3-element subsets yielded by BE algorithm, we obtained Figure 2, which gives us an overall insights into the importance of each level assessed by BE algorithm.

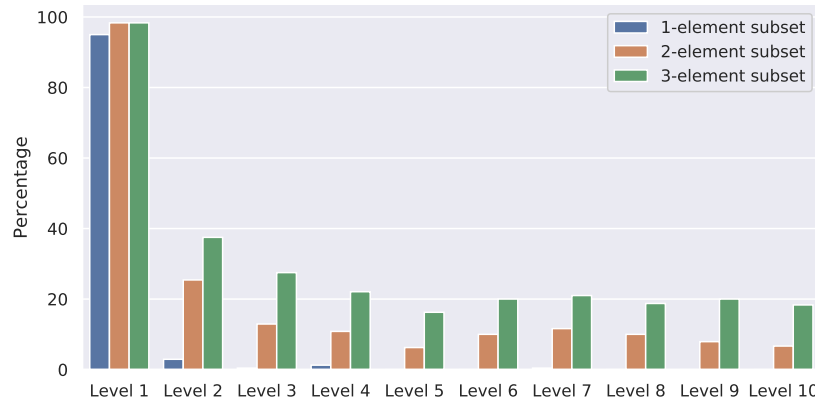


Figure 2: The average percentage of appearance of each level in the 1-element, 2-element and 3-element subsets selected by BE algorithm.

Table 5: The average percentage of appearance in the final solution outputted by BPSO algorithm over all prediction horizons, neural network models and datasets, i.e., average over all columns in Table 2

Level 1	Level 2	Level 3	Level 4	Level 5	Level 6	Level 7	Level 8	Level 9	Level 10
75.83%	16.67%	10.83%	1.67%	4.17%	0.83%	0.83%	0.83%	2.50%	1.67%

From Figure 2, we can see that the top three quote levels of the LOB are also ranked in the same order according to the frequency of being selected by BE. To find potential consensus between BE and BPSO, we also computed the average percentage of appearance in the final solution of BPSO for each level, which is shown in Table 5. From Figure 2 and Table 5, we can see that there is indeed a consensus between BE and BPSO about the ranking between the top three levels while there is no consistent ranking for levels beyond the top three.

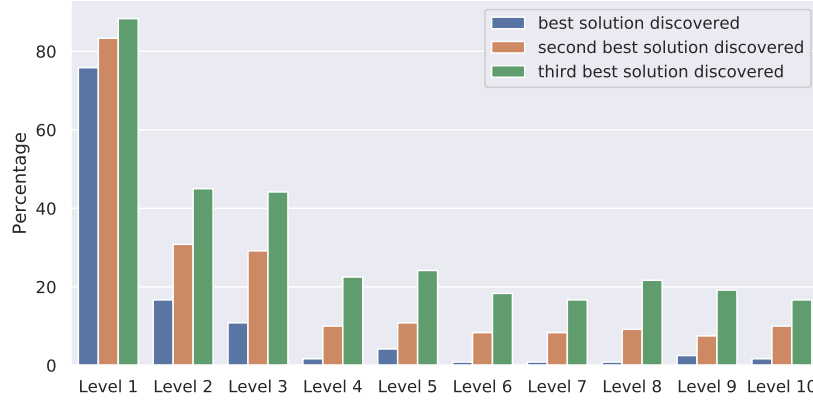


Figure 3: The average percentage of appearance of each level in the best, second best and third best solution discovered by BPSO algorithm during optimization process. The statistics are aggregated from both US and Nordic data for all prediction horizons

For BPSO, we cannot specify the size of the selected subset, thus, we cannot analyse the appearance of a given level with a fixed subset cardinality as in BE. However, for BPSO, we can also look at the average percentage of appearance of each level in the second and third best solutions discovered by the swarm during the course of optimization. The statistics, aggregated from both US and Nordic data in all prediction horizons, are shown in Figure 3, which also confirms the ranking between the top three levels of the LOB suggested by BE.

4.5. Are quotes beyond the best levels needed for good performance?

In the previous analysis, we have identified that the top quotes carry the most important information that is relevant for the prediction of mid-price movements. However, we have not looked at the prediction performances achieved by using the subsets of information selected by the feature selection algorithms. In this section, we analyze this question.

Table 6 shows F1 scores measured on the test set (out-of-sample) of the US and Nordic datasets on Panels A and B, respectively. In particular, the mean F1 and the lowest and highest F1 across the different seed values are provided. The first line of each panel shows the performances of the corresponding neural network models that were trained using data of all ten LOB levels, denoted as **baseline**. The second line of each panel shows the performance with the most important level, sequentially identified with BE.

Table 6: Model performance reported as the mean F1 score in % with the lowest and highest F1 across the different seed values. The results are provided for US (Panel A) and Nordic (Panel B) data sets and for three different prediction horizons, $H = 10, 20, 50$. Moreover, two models for the price prediction with the LOB data are used, **DeepLOB** [25] and **DeepLOB** [22]. The first line shows the performances of the models that were trained using data of all ten LOB levels, denoted as **Baseline**. The second line shows the results on the most important level, sequentially identified with **BE**. The third level shows performance of the models based on the use of the set of levels, optimized by **BPSO**.

H	DeepLOB			TABL		
	10	20	50	10	20	50
Panel A: US Data						
Baseline	65.01 $\pm$ 00.36	66.27 $\pm$ 00.12	65.73 $\pm$ 00.21	64.31 $\pm$ 00.28	65.26 $\pm$ 00.16	65.38 $\pm$ 00.07
BE	63.66 $\pm$ 02.57	60.85 $\pm$ 04.43	64.98 $\pm$ 00.12	60.57 $\pm$ 00.84	63.13 $\pm$ 00.57	64.02 $\pm$ 00.16
BPSO	64.35 $\pm$ 00.22	65.74 $\pm$ 00.09	62.26 $\pm$ 03.69	62.77 $\pm$ 00.87	64.38 $\pm$ 00.44	62.94 $\pm$ 03.17
Panel B: Nordic Data						
Baseline	69.92 $\pm$ 01.86	71.53 $\pm$ 02.22	71.83 $\pm$ 00.49	70.10 $\pm$ 01.23	71.32 $\pm$ 00.89	71.89 $\pm$ 00.74
BE	63.77 $\pm$ 08.12	68.24 $\pm$ 06.78	71.87 $\pm$ 00.08	66.84 $\pm$ 02.02	69.44 $\pm$ 01.35	71.29 $\pm$ 00.72
BPSO	68.53 $\pm$ 01.89	68.18 $\pm$ 03.36	69.07 $\pm$ 03.27	67.06 $\pm$ 02.43	70.38 $\pm$ 01.12	70.15 $\pm$ 03.00

The third level shows the performance with the set of levels, optimized by **BPSO**.

It is clear from the both panels that there are performance degradations with both **BE** and **BPSO** almost under all configurations (the only exception is **BPSO** with $H = 1$ with **DeepLOB** on Nordic data). In particular, when using the subsets of data, i.e. the most informative LOB levels, selected by **BE**, we observed on average 3.76% decline in performance for US stocks, and 3.58% for Nordic stocks. Two conclusions can be drawn: On the first hand, the use of the most informative LOB level data, which, according to Table 1, most often come from the top of the book, provides surprisingly good performance. On the second hand, using quote data across the order-book provides extra performance gains. For the levels that maximize the fitness function within the training procedure with **BPSO**, the average reductions are 2.43% and 3.10% for US and Nordic stocks, respectively. This means that the the optimal selection of the set of LOB levels in the training phase does not lead to the superior out-of-sample performance.

Are these a few percents differences are substantial or marginal? To answer this question, one needs to judge whether improvements can outweigh the extra computational cost, which depend on a specific targets and application settings in practice. Nevertheless, based on our analysis, one can be assured that there is indeed useful information contained in levels beyond

the top quotes in the LOB.

5. Conclusion

The major part of the existing literature uses the top-quotes of the limit order book data only, which makes us to ask how informative the top quotes are compared the quotes beyond them. Moreover, one can ask if multi-level order book data should be used instead of best-level data only, even at the cost of complexity of the model. In this paper, we answered these questions in the context of stock price prediction. Because the financial market mechanisms can be highly non-linear, in contrast to existing papers, we employed a non-linear approach. For a robustness of the results, we used two well-performing machine learning models [25, 22]. Moreover, two data sets from different kind of markets, namely from US and Nordic markets, are used. The questions are addressed by two (variable) feature selection algorithms: one which removes the most irrelevant methods and another one which identifies the optimal set of order book levels for the prediction task: *Backward Elimination* (BE) [45, 46] and *Binary Particle Swarm Optimization* (BPSO) [47]. Furthermore, for additional robustness checks, different prediction horizons are used.

We provide a number of results that are robust across several the dimensions (ML prediction models, feature selection algorithms, data sets, prediction horizons). The best bid and ask levels are systematically identified not only to be the most informative levels in the order books, but to carry most of the information needed for good prediction performance. More specifically, in terms of the models' predictive power (performance), the inclusion of multiple levels from the limit order books improves models' performance by around 3.5%. Thus, even if the top-of-the-book levels contain most of the relevant information, to maximize models' performance one should use all data across all the levels. Moreover, the informativeness of the levels clearly decreases from the first to the fourth level while the rest of the levels are approximately equally important.

6. Acknowledgements

The authors wish to acknowledge CSC – IT Center for Science, Finland, for computational resources. A. Iosifidis acknowledges funding from the Independent Research Fund Denmark project DISPA (Project Number: 9041-00004).

References

- [1] L. E. Harris, V. Panchapagesan, The information content of the limit order book: evidence from nyse specialist trading decisions, *Journal of Financial Markets* 8 (1) (2005) 25–67.
- [2] M. Siikanen, J. Kanninen, J. Valli, Limit order books and liquidity around scheduled and non-scheduled announcements: Empirical evidence from nasdaq nordic, *Finance Research Letters* 21 (2017) 264–271.
- [3] M. Siikanen, J. Kanninen, A. Luoma, What drives the sensitivity of limit order books to company announcement arrivals?, *Economics Letters* 159 (2017) 65–68.
- [4] Y. Aït-Sahalia, P. A. Mykland, L. Zhang, Ultra high frequency volatility estimation with dependent microstructure noise, *Journal of Econometrics* 160 (1) (2011) 160–175.
- [5] F. M. Bandi, J. R. Russell, Separating microstructure noise from volatility, *Journal of Financial Economics* 79 (3) (2006) 655–692.
- [6] A. Dufour, R. F. Engle, Time and the price impact of a trade, *The Journal of Finance* 55 (6) (2000) 2467–2498.
- [7] R. F. Engle, A. J. Patton, Impacts of trades in an error-correction model of quote prices, *Journal of Financial Markets* 7 (1) (2004) 1–25.
- [8] J.-P. Bouchaud, Y. Gefen, M. Potters, M. Wyart, Fluctuations and response in financial markets: the subtle nature of ‘random’ price changes, *Quantitative finance* 4 (2) (2004) 176–190.
- [9] Z. Eisler, J.-P. Bouchaud, J. Kockelkoren, The price impact of order book events: market orders, limit orders and cancellations, *Quantitative Finance* 12 (9) (2012) 1395–1419.
- [10] F. Guilbaud, H. Pham, Optimal high-frequency trading with limit and market orders, *Quantitative Finance* 13 (1) (2013) 79–94.
- [11] T. Hendershott, C. M. Jones, A. J. Menkveld, Does algorithmic trading improve liquidity?, *The Journal of finance* 66 (1) (2011) 1–33.

- [12] A. P. Chaboud, B. Chiquoine, E. Hjalmarsson, C. Vega, Rise of the machines: Algorithmic trading in the foreign exchange market, *The Journal of Finance* 69 (5) (2014) 2045–2084.
- [13] R. Cont, S. Stoikov, R. Talreja, A stochastic model for order book dynamics, *Operations research* 58 (3) (2010) 549–563.
- [14] R. Cont, A. De Larrard, Price dynamics in a markovian limit order market, *SIAM Journal on Financial Mathematics* 4 (1) (2013) 1–25.
- [15] A. Anand, S. Chakravarty, T. Martell, Empirical evidence on the evolution of liquidity: Choice of market versus limit orders by informed and uninformed traders, *Journal of Financial Markets* 8 (3) (2005) 288–308.
- [16] J. T. Linnainmaa, Do limit orders alter inferences about investor performance and behavior?, *The Journal of Finance* 65 (4) (2010) 1473–1506.
- [17] A. Pardo, R. Pascual, On the hidden side of liquidity, *The European Journal of Finance* 18 (10) (2012) 949–967.
- [18] J. D. Farmer, P. Patelli, I. I. Zovko, The predictive power of zero intelligence in financial markets, *Proceedings of the National Academy of Sciences* 102 (6) (2005) 2254–2259.
- [19] N. Hautsch, R. Huang, The market impact of a limit order, *Journal of Economic Dynamics and Control* 36 (4) (2012) 501–522.
- [20] F. Abergel, A. Jedidi, A mathematical approach to order book modeling, *International Journal of Theoretical and Applied Finance* 16 (05) (2013) 1350025.
- [21] M. Dixon, D. Klabjan, J. H. Bang, Classification-based financial markets prediction using deep neural networks, *Algorithmic Finance* 6 (3-4) (2017) 67–77.
- [22] D. T. Tran, A. Iosifidis, J. Kannianen, M. Gabbouj, Temporal attention-augmented bilinear network for financial time-series data analysis, *IEEE transactions on neural networks and learning systems* 30 (5) (2018) 1407–1418.

- [23] J. Sirignano, R. Cont, Universal features of price formation in financial markets: perspectives from deep learning, *Quantitative Finance* 19 (9) (2019) 1449–1459.
- [24] J. A. Sirignano, Deep learning for limit order books, *Quantitative Finance* 19 (4) (2019) 549–570.
- [25] Z. Zhang, S. Zohren, S. Roberts, Deeplob: Deep convolutional neural networks for limit order books, *IEEE Transactions on Signal Processing* 67 (11) (2019) 3001–3012.
- [26] Y. Mäkinen, J. Kanninen, M. Gabbouj, A. Iosifidis, Forecasting jump arrivals in stock prices: new attention-based network architecture using limit order book data, *Quantitative Finance* 19 (12) (2019) 2033–2050.
- [27] P. Nousi, A. Tsantekidis, N. Passalis, A. Ntakaris, J. Kanninen, A. Tefas, M. Gabbouj, A. Iosifidis, Machine learning for forecasting mid-price movements using limit order book data, *IEEE Access* 7 (2019) 64722–64736.
- [28] N. Passalis, A. Tefas, J. Kanninen, M. Gabbouj, A. Iosifidis, Deep adaptive input normalization for time series forecasting, *IEEE Transactions on Neural Networks and Learning Systems* Forthcoming (2019). doi:10.1109/TNNLS.2019.2944933.
- [29] A. Briola, J. Turiel, R. Marcaccioli, T. Aste, Deep reinforcement learning for active high frequency trading, *arXiv preprint arXiv:2101.07107* (2021).
- [30] C. Cao, O. Hansch, X. Wang, The information content of an open limit-order book, *Journal of Futures Markets: Futures, Options, and Other Derivative Products* 29 (1) (2009) 16–41.
- [31] R. Pascual, D. Veredas, What pieces of limit order book information do are informative? an empirical analysis of a pure order-driven market (2003).
- [32] D. A. Hsieh, Chaos and nonlinear dynamics: application to financial markets, *Journal of Finance* 46 (5) (1991) 1839–1877.

- [33] C. P. Chen, Z. Liu, S. Feng, Universal approximation capability of broad learning system and its structural variations, *IEEE transactions on neural networks and learning systems* 30 (4) (2018) 1191–1204.
- [34] W. Long, Z. Lu, L. Cui, Deep learning-based feature engineering for stock price movement prediction, *Knowledge-Based Systems* 164 (2019) 163–173.
- [35] Z. Fu, W. Xu, R. Hu, G. Long, J. Jiang, Mhler-encoder: Modelling the high-frequency changes across stocks, *Knowledge-Based Systems* 224 (2021) 107092.
- [36] H. Gunduz, Y. Yaslan, Z. Cataltepe, Intraday prediction of borsa istanbul using convolutional neural networks and feature correlations, *Knowledge-Based Systems* 137 (2017) 138–148.
- [37] T. Zhou, S. Gao, J. Wang, C. Chu, Y. Todo, Z. Tang, Financial time series prediction using a dendritic neuron model, *Knowledge-Based Systems* 105 (2016) 214–224.
- [38] H. K. Cao, H. K. Cao, B. T. Nguyen, Delafo: An efficient portfolio optimization using deep neural networks, in: *Pacific-Asia Conference on Knowledge Discovery and Data Mining*, Springer, 2020, pp. 623–635.
- [39] Z. Zhang, S. Zohren, S. Roberts, Deep learning for portfolio optimization, *The Journal of Financial Data Science* 2 (4) (2020) 8–20.
- [40] P. M. Addo, D. Guegan, B. Hassani, Credit risk analysis using machine and deep learning models, *Risks* 6 (2) (2018) 38.
- [41] M. Leo, S. Sharma, K. Maddulety, Machine learning in banking risk management: A literature review, *Risks* 7 (1) (2019) 29.
- [42] J. Cao, J. Chen, J. C. Hull, Z. Poulos, Deep hedging of derivatives using reinforcement learning, Available at SSRN 3514586 (2019).
- [43] J. Du, M. Jin, P. N. Kolm, G. Ritter, Y. Wang, B. Zhang, Deep reinforcement learning for option replication and hedging, *The Journal of Financial Data Science* 2 (4) (2020) 44–57.
- [44] G. Chandrashekar, F. Sahin, A survey on feature selection methods, *Computers & Electrical Engineering* 40 (1) (2014) 16–28.

- [45] M. Efroymson, Multiple regression analysis, *Mathematical methods for digital computers* (1960) 191–203.
- [46] R. R. Hocking, A biometrics invited paper. the analysis and selection of variables in linear regression, *Biometrics* (1976) 1–49.
- [47] J. Kennedy, R. C. Eberhart, A discrete binary version of the particle swarm algorithm, in: *1997 IEEE International conference on systems, man, and cybernetics. Computational cybernetics and simulation*, Vol. 5, IEEE, 1997, pp. 4104–4108.
- [48] D. E. Rumelhart, G. E. Hinton, R. J. Williams, Learning representations by back-propagating errors, *nature* 323 (6088) (1986) 533–536.
- [49] D. P. Kingma, J. Ba, Adam: A method for stochastic optimization, *arXiv preprint arXiv:1412.6980* (2014).
- [50] C. Szegedy, W. Liu, Y. Jia, P. Sermanet, S. Reed, D. Anguelov, D. Erhan, V. Vanhoucke, A. Rabinovich, Going deeper with convolutions, in: *Proceedings of the IEEE conference on computer vision and pattern recognition*, 2015, pp. 1–9.
- [51] S. Hochreiter, J. Schmidhuber, Long short-term memory, *Neural computation* 9 (8) (1997) 1735–1780.
- [52] K. Cho, B. Van Merriënboer, C. Gulcehre, D. Bahdanau, F. Bougares, H. Schwenk, Y. Bengio, Learning phrase representations using rnn encoder-decoder for statistical machine translation, *arXiv preprint arXiv:1406.1078* (2014).
- [53] V. F. Flack, P. C. Chang, Frequency of selecting noise variables in subset regression analysis: a simulation study, *The American Statistician* 41 (1) (1987) 84–86.
- [54] S. Derksen, H. J. Keselman, Backward, forward and stepwise automated subset selection algorithms: Frequency of obtaining authentic and noise variables, *British Journal of Mathematical and Statistical Psychology* 45 (2) (1992) 265–282.

- [55] P. C. Austin, J. V. Tu, Automated variable selection methods for logistic regression produced unstable models for predicting acute myocardial infarction mortality, *Journal of clinical epidemiology* 57 (11) (2004) 1138–1146.
- [56] A. Ntakaris, M. Magris, J. Kanninen, M. Gabbouj, A. Iosifidis, Benchmark dataset for mid-price forecasting of limit order book data with machine learning methods, *Journal of Forecasting* 37 (8) (2018) 852–866.
- [57] D. T. Tran, J. Kanninen, M. Gabbouj, A. Iosifidis, Data normalization for bilinear structures in high-frequency financial time-series, in: *International Conference on Pattern Recognition (ICPR)*, 2020.
- [58] S. Ioffe, C. Szegedy, Batch normalization: Accelerating deep network training by reducing internal covariate shift, in: *International conference on machine learning*, PMLR, 2015, pp. 448–456.
- [59] D. Ulyanov, A. Vedaldi, V. Lempitsky, Instance normalization: The missing ingredient for fast stylization, *arXiv preprint arXiv:1607.08022* (2016).
- [60] K. He, X. Zhang, S. Ren, J. Sun, Delving deep into rectifiers: Surpassing human-level performance on imagenet classification, in: *Proceedings of the IEEE international conference on computer vision*, 2015, pp. 1026–1034.